

On Approximating (Connected) 2-Edge Dominating Set by a Tree

Toshihiro Fujito¹  · Tomoaki Shimoda¹

Published online: 3 April 2017

© Springer Science+Business Media New York 2017

Abstract The *edge dominating set* problem (EDS) is to compute a minimum size edge set such that every edge is dominated by some edge in it. This paper considers a variant of EDS with extensions of multiple and connected dominations combined. In the *b*-EDS problem, each edge needs to be dominated *b* times. CONNECTED EDS requires an edge dominating set to be connected while it has to form a tree in TREE COVER. Although each of EDS, *b*-EDS, and CONNECTED EDS (or TREE COVER) has been well studied, each known to be approximable within 2 (or $8/3$ for *b*-EDS in general), nothing is known when these extensions are imposed simultaneously on EDS unlike in the case of the (vertex) dominating set problem. We consider CONNECTED 2-EDS and 2-TREE COVER (i.e., a combination of 2-EDS and TREE COVER), and present a polynomial algorithm approximating each within 2. Moreover, it will be shown that the single tree computed is no larger than twice the optimum for (not necessarily connected) 2-EDS, thus also approximating 2-EDS equally well. It also implies that 2-EDS with clustering properties can be approximated within 2 as well.

Keywords Edge dominating sets · Approximation algorithms · Tree cover · Connected dominating sets · *b*-edge domination

A preliminary version of this work appeared in Proc. 11th CSR (International Computer Science Symposium in Russia), 2016. This work is supported in part by JSPS KAKENHI under Grant Number 26330010.

✉ Toshihiro Fujito
fujito@cs.tut.ac.jp

¹ Department of Computer Science and Engineering, Toyohashi University of Technology, Toyohashi 441-8580, Japan

1 Introduction

In an (undirected) graph $G = (V, E)$ a vertex is said to *dominate* itself and all the vertices adjacent to it, and a vertex set $S \subseteq V$ is a *dominating set* for G if every vertex in G is dominated by some in S . The problem of computing a minimum size dominating set is called DOMINATING SET (DS). Domination is one of the most fundamental concepts in graph theory with numerous applications to a variety of areas [1, 10]. The two books [24, 25] contain the main results and applications of domination in graphs.

Many variants of the basic concepts of domination have appeared in the literature, and two of them are relevant to the current work. One is *multiple domination*, introduced by Fink and Jacobson [18, 19] with an obvious motivation of fault tolerance and/or robustness, and in fact there exist various types of multiple domination; k -*domination* requires every vertex *not* in $S \subseteq V$ to be dominated by k members of S while k -*tuple domination* requires every vertex in G to be dominated by k members of S (A survey on combinatorial properties of theirs and other multiple dominations can be found in [8]). Another well-known variant of DS demands connectivity on dominating sets, and $S \subseteq V$ is a *connected dominating set* for G if S is a dominating set for G inducing a connected subgraph of G . The problem CONNECTED DOMINATING SET has been studied in graph theory for many years, and recently it has become a hot topic due to its application in wireless networks for virtual backbone construction [7, 12, 13]. Moreover, a unifying generalization of multiple and connected dominations has recently received considerable attention, leading to the concept of k -*connected m -dominating set* problem [11]. A minimum vertex set $S \subseteq V$ is sought here such that S is an m -dominating set and simultaneously the subgraph $G[S]$ induced by S is k -connected, where various approximation results are becoming available mainly for unit-disk graphs [22, 27] as well as for general graphs [33, 38].

In this paper we focus on the algorithmic aspects of EDGE DOMINATING SET (EDS), which is yet another natural variant of DS, and its generalizations. Here, an edge is said to *dominate* itself and all the edges incident to it, and an edge set $F \subseteq E$ is an *edge dominating set* (*eds*) for G if every edge in G is dominated by some in F . The multiple domination variant of EDS is known as the b -EDGE DOMINATING SET (b -EDS) problem, and here, $F \subseteq E$ is a *b -edge dominating set* (*b -eds*) if every edge in G is dominated by b members of F (and hence, it corresponds to the b -tuple DS on line graphs). The CONNECTED EDGE DOMINATING SET problem is a connected variant of EDS, and it is also known as TREE COVER, the problem of covering all the edges in G by the vertex set of a tree, as any connected eds is necessarily cycle-free if it is minimal. Despite the fact that EDS is equivalent to the special case of DS with G restricted to line graphs, EDS seems to have its own history of being a topic of research. Since it was shown to be NP-complete by Yannakakis and Gavril [37], the computational complexity of EDS has been well explored. Computing any maximal matching is a 2-approximation to EDS since it is equivalent to the problem of computing the *minimum maximal matching* [23]. No better approximation has been found in the general case although EDS admits a PTAS for some special cases [4, 26], and some nontrivial approximation lower bounds have been derived (under some likely complexity hypothesis) [9, 14, 31]. Chlebik and Chlebikova proved that it is

NP-hard to approximate EDS within any factor better than $7/6$ [9], and Escoffier et al. gave an improved hardness result by showing that it is not approximable within $5\sqrt{5} - 10 + \epsilon < 1.18$ in polynomial time unless $P=NP$ [14]. Assuming the unique game conjecture (UGC), some inapproximability results on dense instances were shown in [31], a corollary of which states that EDS is inapproximable within ratio $3/2 - \epsilon$ for every $\epsilon > 0$ (under UGC). The parameterized complexity of EDS has also been extensively studied [6, 14, 15, 20, 36]. The general case of b -EDS is known to be approximable within $8/3$ [5], but within 2 for $b \leq 3$ [21], and Connected EDS can be within 2 [2, 30]. Interestingly though, to the best of our knowledge, no counterpart of the k -connected m -dominating set problem for EDS has appeared in the literature, and nothing is known for Connected b -EDS for $b \geq 2$.

1.1 Our Work

Let us assume henceforth that input graphs G are connected. The main subject of the paper is the CONNECTED 2-EDS problem (C2-EDS); that is, the problem of computing a minimum 2-eds inducing a connected subgraph. Another way of imposing the connectivity on b -EDS is to enforce a solution to be a subgraph of certain types in the spirit of Arkin et al. [2]. It is natural to define 2-TREE COVER (2-TC) as the problem of computing a smallest tree, termed a *2-tree cover*, in G such that its edge set forms a 2-eds for G . Notice that a spanning tree is always a 2-tree cover (assuming G is a feasible instance) and that C2-EDS and 2-TC are not equivalent as an optimal solution for the former is not necessarily a tree.

To explain the contributions of the current work and their implications, consider the following natural integer program formulation of 2-EDS, the version where solutions are not required to be connected:

$$\min \{x(E) \mid x(\delta(e)) \geq 2 \text{ and } x_e \in \{0, 1\}, \forall e \in E\},$$

where $x(F) = \sum_{e \in F} x_e$ for $F \subseteq E$, and $\delta(e) = \{e\} \cup \{e' \in E \mid e' \text{ is adjacent to } e\}$ for $e \in E$. Replacing the integrality constraints by linear constraints $0 \leq x_e$ results in the LP and its dual in the following forms:

$$\begin{array}{ll} \text{LP:(P)} & \min z_P(x) = x(E) \\ \text{subject to:} & x(\delta(e)) \geq 2, \quad \forall e \in E \\ & x_e \geq 0, \quad \forall e \in E \end{array} \qquad \begin{array}{ll} \text{LP:(D)} & \max z_D(y) = \sum_{e \in E} 2y_e \\ \text{subject to:} & y(\delta(e)) \leq 1, \quad \forall e \in E \\ & y_e \geq 0, \quad \forall e \in E \end{array}$$

Let $\text{dual}_2(G)$ denote the optimal value of LP:(D) above for graph G . The main contribution of the paper is to present an approximation algorithm named C2EDS for 2-TC such that 1) it computes a tree the edge set of which is a 2-eds for G , and 2) the tree computed is of size no larger than $2 \cdot \text{dual}_2(G)$.

Theorem 1 *Given a connected graph G , the algorithm C2EDS outputs a tree, dominating every edge at least twice, of size bounded by $2 \cdot \text{dual}_2(G) - 1$.*

Since $\text{dual}_2(G)$ lower bounds the minimum size of 2-eds, it follows that 2-TC as well as C2-EDS can be approximated within 2. The theorem also implies in turn that 2-EDS can be approximated by a tree within 2, and hence,

Corollary 1 *The algorithm C2EDS computes a tree approximating 2-EDS, C2-EDS, and 2-TC each within a factor of 2.*

It is also worth pointing out that, considering the case of G being a complete graph, the gap of $\text{LP}(\text{P})$ from the integral optimum for C2-EDS can be arbitrarily close to 2. Moreover, tight instances showing that ratio of 2 is best possible for the given algorithm will be presented at the end of Section 4. As mentioned earlier, satisfying both connectivity and multiple domination has been already considered in the case of the vertex domination. It then tends to be harder to approximate, however, even for unit-disk graphs [32, 34, 35], than when satisfying either one only, in good contrast with the case of EDS as shown in this paper.

A vertex set $C \subseteq V$ is a *vertex cover* (vc) if every edge in G is incident to some member of C , a *connected vertex cover* if it is a vc inducing a connected subgraph, and it is called a *t -total vertex cover* (t -tvc) ($t \geq 1$) if it is a vc such that each connected component of the subgraph induced by C has at least t vertices. Hence, if C is a t -tvc, each member of the vertex cover C belongs to a “cluster” containing at least t members of C . Having such clustering properties could be desirable or required in some applications, and variants with such properties enforced are considered in other combinatorial optimization problems as well, such as *r -gatherings* [3]. The problem t -TVC of computing a minimum t -tvc was introduced in [17, 29], and was further studied in [16], where t -TVC was shown to be approximable within 2 for each $t \geq 1$ [17]. This is based on the Savage’s algorithm computing a connected vc of size no larger than twice the minimum vc size [30]. Our algorithm reveals the existence of a similar phenomenon between 2-EDS and C2-EDS, and when 2-EDS with clustering properties is considered, the above result implies that it can be approximated within 2 for any cluster size $t (\geq 1)$ (Note: No such relation is known between EDS and CEDS).

2 Preliminaries

For a vertex set $U \subseteq V$, $\Gamma(U)$ denotes the set of all the vertices in G which are adjacent to at least one vertex of U (i.e., $\Gamma(U) = \{v \in V \mid \{u, v\} \in E \text{ for some } u \in U\}$), and $\Gamma(u)$ means $\Gamma(\{u\})$. For a vertex $u \in V$ let $\delta(u)$ denote the set of edges incident to u . The degree of a vertex u is denoted by $\deg(u)$. An *empty* tree is meant to be one consisting of a single node (and no edge).

2.1 Gallai-Edmonds Decomposition

For any graph G denote by D the set of vertices v in G such that there exists a maximum matching of G not spanning v . Let A denote the set of vertices in $V - D$ adjacent to at least one vertex in D , and let $C = V - A - D$. A graph G is called

factor-critical if the removal of any vertex from G results in a graph having a perfect matching. For a bipartite graph G with bipartition (X, Y) , the *surplus of a set* $W \subseteq X$ is defined by $|\Gamma(W)| - |W|$, and the *surplus of the bipartite graph* G is the minimum surplus of non-empty subsets of X .

This decomposition, which can be computed in polynomial time via the Edmonds matching algorithm, provides important information concerning all the maximum matchings in G :

Theorem 2 (the Gallai-Edmonds structure theorem. See [28])

1. *The components of the subgraph induced by D are factor-critical.*
2. *The subgraph induced by C has a perfect matching.*
3. *The bipartite graph obtained from G by deleting the vertices of C and the edges in $G[A]$ and by contracting each component of D to a single vertex has positive surplus (as viewed from A).*

Call a vertex in A an *a-node* and one in D a *d-node*. The set of *a-nodes* in a subgraph H of G is denoted by $A(H)$ and that of *d-nodes* by $D(H)$.

3 Algorithm Design Overview

One possible way to relate 2-EDS and matchings is to observe that $(1/2)y_M$ is dual feasible to $\text{LP}:(D)$ for any matching $M \subseteq E$ and the incidence vector $y_M \in \{0, 1\}^E$ of M , which implies that the minimum 2-eds size is lower bounded by $|M|$ (or the minimum b -eds size in general lower bounded by $b|M|/2$). Therefore, any 2-eds of size bounded by $2|M|$ for some matching M is a 2-approximation to 2-EDS, and in fact such a 2-eds always exists [21]. When it has to be additionally connected, however, such a graph can be easily found where any connected 2-eds is larger than $2|M|$ even if M is a maximum matching¹, and we take the following measures to deal with this; 1) locate and identify subgraphs of G , with the help of the Gallai-Edmonds decomposition of G , where matching based duals do not work, and 2) devise more elaborate dual assignments for such subgraphs. The algorithm is thus designed, based on the G-E decomposition of G , to compute a forest $\mathcal{F}$ of trees in G s.t.

Property 1. $\mathcal{F}$ spans $A \cup C$ entirely and all the nontrivial components of $G[D]$.

Property 2. Every *a-node* has degree of at least 2 (i.e., non-leaf) in $\mathcal{F}$.

The nodes in G not spanned by $\mathcal{F}$ then are singletons in $G[D]$ only (by Property 1), and they are adjacent only to *a-nodes*, each of which is of degree at least 2 in $\mathcal{F}$ (by Property 2). It thus follows that a feasible 2-eds for G can be computed by

¹Consider for instance a star S_n with the center node u having n leaves, and compose a graph G by attaching a distinct triangle to each leaf of S_n . Then, a connected 2-eds needs to include all the edges of S_n and at least 2 edges from each triangle, totaling to $3n$ edges. A maximum matching on the other hand is of size $n + 1$, consisting of one edge (u, v) in S_n , the edge not incident to v from the triangle attached to v , and one edge from each of the remaining triangles.

minimally connecting all the trees in $\mathcal{F}$ into a single tree. In fact it is relatively easy to find a forest satisfying the properties above for the following reasons. To defer the consideration on how to span nontrivial components of $G[D]$, suppose each of them is shrunk into a single d -node, which we call a *fat d -node*, and let G' denote the resulting graph. It can be induced from Theorem 2.3 that $G'[A \cup D]$ contains a forest such that every node in A has degree 2 in it (see [28]). Thus, by additionally using a tree spanning each component of $G[C]$ (and trees spanning nontrivial components of $G[D]$ at the end), a forest satisfying both Properties 1 and 2 can be obtained.

We are also concerned with performance guarantee of the algorithm, and we consider additionally imposing the following two properties on $\mathcal{F}$ for that purpose:

- Property 3. $\mathcal{F}$ is “compact” enough that all the trees can be glued together, using only extra edges (and no extra nodes).
- Property 4. There exists a dual feasible $y \in \mathbb{R}^E$ s.t. $y(E[T]) \geq (|E(T)| + 1)/4$ for each $T \in \mathcal{F}$, where $E[T]$ denotes the set of edges induced by $V(T)$.

Suppose a tree T_{fin} is constructed by minimally connecting all the trees in $\mathcal{F}$, and to do so, it suffices to use $(|E(T)| + 1)$ edges per tree $T \in \mathcal{F}$ due to Property 3. The total number of edges in T_{fin} is thus no larger than

$$\sum_{T \in \mathcal{F}} (|E(T)| + 1) \leq \sum_{T \in \mathcal{F}} 4y(E[T]) \leq \sum_{e \in E} 4y_e \leq 2 \cdot \text{dual}_2(G),$$

for some dual feasible $y \in \mathbb{R}^E$, where the first inequality is due to Property 4, implying that this is a 2-approximation algorithm for 2-EDS.

Our main goal is thus to devise a way to compute a forest $\mathcal{F}$ satisfying Properties 3 and 4 on top of 1 and 2. It is noted here that Property 3 follows easily when T (or the component T spans) has a perfect matching M , by assigning $1/4$ to y_e for each edge $e \in M$. For this and some other reasons to be explained later, it is not so hard to deal with the components in $G[C]$ as well as the nontrivial components in $G[D]$, and much of the work will be spent in computing trees in $G'[A \cup D]$.

3.1 Two Types of Trees

To ensure Property 2 in building trees in $G'[A \cup D]$, we consider a path (u, v, w) of length 2 with an a -node v in the middle, as one unit, and call it a *2-edge*. A tree composed of 2-edges only, with any 2-edge (u, v, w) treated as an “edge” $\{u, w\}$, is called a *2-tree*.

Basic Trees A 2-edge (u, v, w) is called a (*dad*)-edge when both of u and w are d -nodes. A 2-tree consisting of (*dad*)-edges only is called *basic*. Clearly, for any basic tree T , every a -node is of degree 2 in T , and all the leaves of T are d -nodes.

A basic tree T is *maximal* if no node of T is adjacent to an a -node outside of T ; that is, if $\Gamma(V(T)) \setminus V(T) \subseteq D$. It will be shown that maximal basic trees can be

efficiently computed in $G'[A \cup D]$, and as will be seen, maximal basic trees constitute basic building blocks in construction of a forest in $G'[A \cup D]$.

Bridged Trees Recall that D is an independent set in G' (as nontrivial components are shrunk to singletons) while A is not in general, and there could be edges among a -nodes. Therefore, there could be two disjoint basic trees and an edge connecting them at an a -node in each of them, for instance, and they can never be maximal basic trees. To avoid such situation we introduce an additional type of tree comprised of 2-edges.

A 2-edge (u, v, w) is called a (*aad*)-edge when one of u and w is an a -node and the other is a d -node. A path $P = (d_1, a_1, a_2, d_2)$ of length 3 is called a *bridge-path* when both of the end-nodes d_1 and d_2 are d -nodes and both of the mid-nodes a_1 and a_2 are a -nodes. A tree T constructed by starting with a bridge-path and inductively extending T by attaching either a (*dad*)-edge or an (*aad*)-edge is called a *bridged tree*, where an (*aad*)-edge is always attached only at an a -node of T .

As in a basic tree, an a -node cannot occur at a leaf in any bridged tree, from the way it is constructed, and hence, its degree is at least 2 (which can be larger unlike basic trees). Also observe that a bridged tree is perfectly matchable, unlike a basic tree, implying that it possesses Property 4. Once again, a bridged tree T in G is called *maximal* if no node of T is adjacent to an a -node outside of T ; that is, if $\Gamma(V(T)) \setminus V(T) \subseteq D$.

4 Algorithms

4.1 Tree Construction

Let H be a connected component of $G'[A \cup D]$. We describe here how to construct within H either a maximal basic tree or a maximal bridged tree, and outline three procedures used for this purpose, `basicTree( $s$ )`, `basicTree&Path( $s$ )`, and `bridgedTree( $s$ )` (See Section 4.1.1 for more detailed and precise descriptions for them).

1. The procedure `basicTree( $s$ )`, when called with an initial d -node s , returns a basic tree T containing s . It computes T in the DFS fashion, starting with s and using a stack data structure to keep currently active d -nodes as follows. It repeats extending T by a (*dad*)-edge from the currently visited d -node d if there exists an a -node in $\Gamma(d) \setminus A(T)$ while backtracking to a previous d -node if no such a -nodes exist. It differs from the ordinary DFS-tree construction in how to handle a case where the addition of a new edge would introduce a cycle in T . Suppose there exists an unvisited a -node a' in the neighbor of the currently visited d -node d . Let d' be another d -node adjacent to a . If $d' \notin D(T)$, adding the 2-edge (d, a', d') to T properly extends T to a larger basic tree. If $d' \in D(T)$, on the other hand, $T \cup \{(d, a', d')\}$ induces a cycle. Denote this cycle

by B . Then, as $|A(B)| = |D(B)|$, there must exist an edge (a'', d'') s.t. $a'' \in A(B)$ and $d'' \notin D(B)$ due to Theorem 2.3. If $d'' \notin D(T)$ this time, by adding both (d, a', d') and (a'', d'') while removing the cycle (by dropping one edge incident to a''), T can be properly extended to a larger basic tree. Certainly, it could be the case again that $d'' \in D(T)$, and hence, adding (a'', d'') introduces another cycle in T . In such a case, let B denote the union of these cycles, and repeat the same argument and procedure. Eventually such an (ad) -edge would be found, of which the d -node lies outside of T . At that point, T can be properly extended by adding some number of (ad) -edges, on top of the initial (d, a', d') , and removing the same number of (ad) -edges within T . This part will be taken care of by procedure `processCycle`.

The procedure `basicTree(s)` thus extends an edge from any d -node to any neighboring a -node if it is not yet a part of the tree. Therefore, it computes a basic tree T s.t. $\Gamma(D(T)) \subseteq A(T)$. It may not be maximal, however, as there could be an a -node in T adjacent to another a -node outside of T . The next procedure `basicTree&Path(s)` is designed to compute a tree T , starting with a single d -node s , s.t. T is either a maximal basic tree or otherwise, T contains a single bridge-path joining two vertex disjoint basic trees.

2. The procedure `basicTree&Path(s)`, which is a slightly modified `basicTree(s)`, returns a tree T containing s s.t. T is either maximal basic or T contains a single bridge-path joining two vertex disjoint basic trees. It uses a flag `bridged` (initially, `false`) so that T contains at most one bridge-path. It constructs a tree T just like `basicTree(s)` does, but when trying to extend T from a current d -node d , it first checks whether a bridge-path P exists emanating from d (before the existence of a (dad) -edge incident to d), and if exists, it extends T along P , as long as `bridged` is `false` (that is when no bridge-path is yet contained in T). Because the extension via a bridge-path has a higher priority over that via a (dad) -edge, all the nodes in P must have been unvisited except for d when P is found, and the other end of P becomes the next active d -node (while `bridged` is set to `true`).

`basicTree&Path(s)` behaves exactly same as `basicTree(s)` once `bridged` becomes `true`, except that we also need to modify the way to handle possible occurrences of cycles so that the middle edge (a_1, a_2) of a bridge-path $P = (d_1, a_1, a_2, d_2)$ already contained in T , is never deleted (as it would destroy the “bridgedness”). This can be easily handled by treating P as if $P = (d_1, a_1, d_2)$ (or $P = (d_1, a_2, d_2)$ depending on the particular case).

Clearly, `basicTree&Path(s)` returns T such that it is either a basic tree or it contains a single bridge-path joining two vertex disjoint basic trees. Observe that, in the former case, no bridge-path has been found throughout the process of extending T , which implies that no a -node of T is adjacent to another a -node (i.e., $\Gamma(A(T)) \subseteq C \cup D$). Therefore, when T returned by `basicTree&Path(s)` is basic (i.e., `bridged` is `false`), it must be a maximal basic tree.

3. The procedure `bridgedTree(s)` returns a tree T containing a d -node s s.t. T is either a basic or a bridged tree, and maximal one in either case. It first calls

`basicTree&Path(s)`, which returns a tree T . If T is basic, the procedure ends here. Otherwise, it must be a tree consisting of a single bridge-path $P = (d_1, a_1, a_2, d_2)$ joining two vertex disjoint basic trees, T_1 and T_2 . We extend T to a maximal bridged tree. Recall that either T_1 or T_2 is no longer extendable from any of its d -nodes as they are constructed by `basicTree&Path(s)`, but they can possibly be from the a -nodes of them by (aad) -edges. Besides, it might be also possible to further grow T by extending it from each of a_1 and a_2 . Therefore, we first collect all of these a -nodes of T in `midAs`, and check if any unvisited node a' exists in $\Gamma(a)$ for each $a \in \text{midAs}$. If it doesn't, extension from a is complete and a is removed from `midAs`. If it does exist, any d -node d' adjacent to a' must be unvisited, and we undertake the following steps: compute a basic tree T' by calling `basicTree(d')`, extend T by attaching T' via 2-edge (a, a', d') , and add all the a -nodes of T' together with a' to `midAs`. These operations are repeated as long as `midAs` is nonempty, and when the procedure terminates, T must be a maximal bridged tree.

4.1.1 More Details for Tree Construction

The following algorithm `basicTree(s)`, when called after initializing all the nodes marked “unvisited” and the stack data structure S to be empty, constructs a basic tree T , starting with a single d -node s , by repeatedly attaching a (dad) -edge. For any d -node d in T all the a -nodes adjacent to d become a part of T , and hence, T output by the algorithm has the property that $\Gamma(D(T)) \subseteq A(T)$.

Algorithm 1 `basicTree(s)`

1. Initialization:
 - set T to be a tree consisting of s only,
 - mark s “visited”,
 - push s to S .
 2. While $S \neq \emptyset$ do
 - (a) Let $d \leftarrow \text{Top}(S)$.
 - (b) If there exists an unvisited a -node a' in $\Gamma(d)$ then
 - (i) Visit a' by marking a' “visited”.
 - (ii) Pick a (dad) -edge (d, a', d') .
 - (iii) If d' is “unvisited” then extend T from d by attaching the 2-edge (d, a', d') , mark d' “visited”, and push d' to S .
 - (iv) Else call `processCycle((d, a', d'), T)`.
 - (c) Else `pop(S)`.
 3. Output T .
-

To complete the procedure above, we need to describe a way to handle the 2-edge (d, a', d') in Step 2(b)iv, the addition of which would introduce a cycle in T .

Algorithm 2 $\text{processCycle}((\tilde{d}_1, \tilde{a}, \tilde{d}_2), T)$

1. Initialize the subtree $\tilde{T}$ of T to be the path on T between $\tilde{d}_1$ and $\tilde{d}_2$, and let B denote the cycle formed by $\tilde{T}$ and $(\tilde{d}_1, \tilde{a}, \tilde{d}_2)$.
- 2.
3. Since $|A(B)| = |D(B)|$, there must exist an edge (a', d') such that $a' \in A(B)$ and $d' \notin D(B)$; find such an edge.
4. While $d' \in D(T)$ do
 - (a) Extend the subtree $\tilde{T}$ of T by attaching to it the unique shortest path on T between d' and $\tilde{T}$.
 - (b) Since $|A(B \cup \tilde{T})| = |D(B \cup \tilde{T})|$, there must exist an edge (a', d') such that a' is a node of $B \cup \tilde{T}$ and d' lies outside of $B \cup \tilde{T}$; find such an edge.

See an example run of Steps 1 through 3 in Fig. 1. In Fig. 1a, after initialized to the path between $\tilde{d}_1$ and $\tilde{d}_2$, $\tilde{T}$ (depicted as a wiggly line) is extended, due to (a'_1, d'_1) , to the path between $\tilde{d}_1$ and $\tilde{d}_2$. In Fig. 1b $\tilde{T}$ is further extended by (a'_2, d'_2) , after which (a'_3, d'_3) is found with d'_3 not lying on T , and the while-loop of Step 3 is exited.

When the While-loop finishes, the subtree $\tilde{T}$ of T has the following property due to the way it has been constructed: Let a be any a -node in $\tilde{T}$ not lying on B , and split

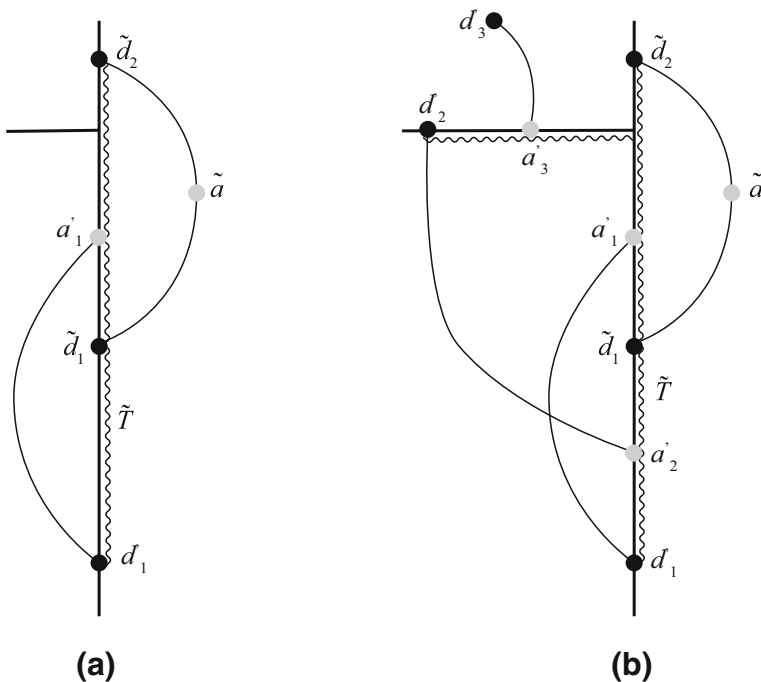


Fig. 1 An example run of Steps 1 through 3 in processCycle

$\tilde{T}$ at a into two trees $\tilde{T}_1$ and $\tilde{T}_2$ where $\tilde{T}_1$ has a non-empty overlap with B while $\tilde{T}_2$ has none. Then, there exists an edge (a', d') outside of T such that $a' \in A(B \cup \tilde{T}_1) \setminus \{a\}$ and $d' \in D(\tilde{T}_2)$.

4. Let (a', d') denote the last edge found by the While-loop (Note: $a' \in A(B \cup \tilde{T})$ and $d' \notin V(T)$). Mark d' “visited” and push d' to S .
5. Set $\tilde{T}_1 = \tilde{T}$ and $\tilde{T}_2 = \emptyset$.
6. While $a' \in A(\tilde{T}_1) \setminus A(B)$ do
 - (a) Let (d_1, a', d_2) be the 2-edge of $\tilde{T}_1$ of which a' is the mid-node.
 - (b) Suppose, between d_1 and d_2 , d_1 is the one closer to B (than d_2 is). Split $\tilde{T}_1$ into two trees by removing the edge (d_1, a') , name the one containing d_1 as $\tilde{T}_1$ anew, and attach the other to $\tilde{T}_2$ by adding the edge (a', d') to it.
 - (c) Do the same operations to T , removal of (d_1, a') and addition of (a', d') , which corresponds to replacing the 2-edge (d_1, a', d_2) in T by (d', a', d_2) .
 - (d) Find a new edge (a', d') such that $a' \in \tilde{T}_1$ and $d' \in \tilde{T}_2$.
7. Let (d_1, a', d_2) be the 2-edge of B of which a' is the mid-node; remove one of (d_1, a') and (a', d_2) from T and add (a', d') to T , which corresponds to replacing the 2-edge (d_1, a', d_2) in T by (d', a', d_2) (or by (d_1, a', d')).
8. Add the 2-edge $(\tilde{d}_1, \tilde{a}, \tilde{d}_2)$ to T .
/* end of processCycle */

See an example run of Steps 4 through 8 above in Fig. 2. In Fig. 2a two edges, (a'_3, d'_3) and (a'_2, d'_2) , have been added to T (and two edges removed from T) in this order. $\tilde{T}_1$ and $\tilde{T}_2$, initially being $= \tilde{T}$ and $= \emptyset$, respectively, have been modified as shown. Figure 2b shows how addition of the 2-edge $(\tilde{d}_1, \tilde{a}, \tilde{d}_2)$ to T is completed in Step 8.

Notice that the existence of the edge (a', d') to be found in Step 6(d) is guaranteed, as long as $\tilde{T}_1$ is not entirely contained in B , by the property of $\tilde{T}$ described above. Once the While-loop finishes, as $a' \in A(B)$, removal of (d_1, a') or (a', d_2) does not split the subgraph $B \cup \tilde{T}_1$, and this completes handling of the cycle B to be introduced in T by the 2-edge $(\tilde{d}_1, \tilde{a}, \tilde{d}_2)$.

To construct a tree T such that T contains a single bridge-path joining two vertex disjoint basic trees, we modify `basicTree` as follows: Introduce a new flag `bridged` (initially set to `false`) and insert the following steps between Steps 2(a) and 2(b) in `basicTree`.

1. If `bridged` is (still) `false` then
 - (a) Check if there exists an unvisited a -node a in $\Gamma(d)$ and an edge between a and another a -node a' .
 - (b) If such an edge (a, a') exists, $P = (d, a, a', d')$ must be a bridge-path for any d -node $d' \in \Gamma(a') \setminus \{d\}$, and in this case,
 - (i) Extend T from d by attaching P .
 - (ii) Mark all of a, a' and d' as visited, and push d' to S .
 - (iii) Set `bridged` to `true`.

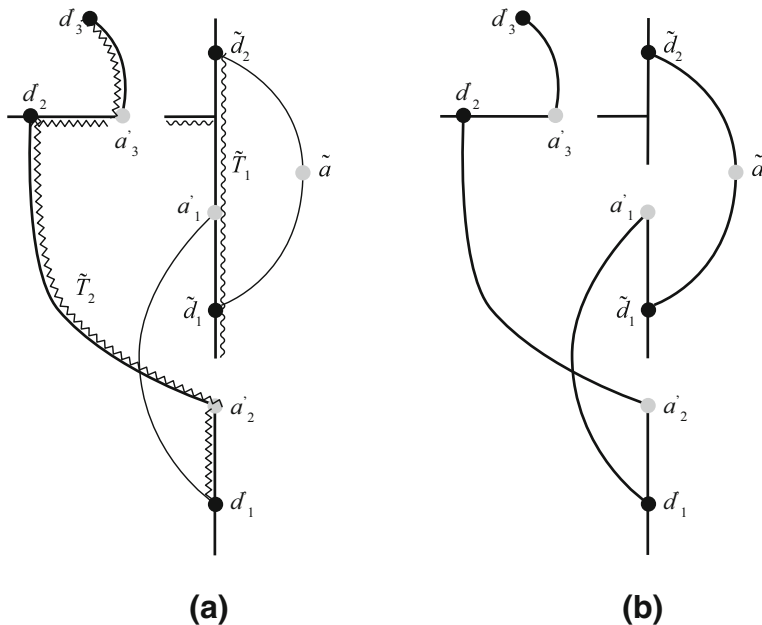


Fig. 2 An example run of Steps 4 through 8 in `processCycle`

We call this algorithm `basicTree&Path(s)`. The flag `bridged` is used so that, once a bridge-path is included in T , the newly introduced steps are subsequently skipped. This way, T may contain at most one bridge-path in it. The reasons why T is constructed in the DFS fashion using the stack data structure and why $P = (d, a, a', d')$ must be a bridge-path as stated in the newly introduced Step 1(b) are as follows:

- When an edge (a, a') is found in Step 1(b) above, `bridged` is still `false` and a' is not yet a part of T for, if it were, it is a mid-node of some (dad) -edge (d_1, a', d_2) since no bridge-path exists in T at this point. It must have been extended either from d_1 or d_2 , say d_1 . But then, before attaching (d_1, a', d_2) to T , the existence of a bridge-path must have been checked in Step 1 above with $d = d_1$ and (d_1, a', a, d) must have been found at that point, contradicting the fact that none had been found earlier.
- Since T is constructed in the DFS fashion, as soon as a d -node d' is inserted into T , it is checked if a bridge-path emanating from d' exists or not. So, when a bridge-path (d, a, a', d') is found in Step 1(b) above, d' cannot be a part of T since, otherwise, it must have been found earlier right after d' became a part of T , contradicting the fact that none had been found earlier.

We also need to modify `processCycle(B, T)` so that the middle edge of a bridge-path $P = (d_1, a_1, a_2, d_2)$ is never deleted by the procedure, and we can avoid this occurrence by minor modifications to `processCycle(B, T)`. Depending on

whether P in T is contained in the cycle B or not, the ways to handle P are slightly different:

- Case P is contained in B . Initialize P to (d_1, a_1, d_2) by bypassing a_2 and removing all the edges incident to a_2 . If one of (d_1, a_1) and (a_1, d_2) needs to be removed in Step 8 of `processCycle` (B, T), choose (d_1, a_1) to be removed. This way, the middle edge (a_1, a_2) can always be recovered when the procedure is over.
- Case P is not contained in B . Let d_1 be the one, between d_1 and d_2 , closer to B within T , and initialize P to (d_1, a_1, d_2) as in the previous case (the other case is similarly handled). In this way, even if P gets involved in Steps 6(a) through (c), only (d_1, a_1) is removed and (a_1, a_2) can be recovered when the procedure is over.

The next procedure builds a maximal tree, basic or bridged, from a single d -node in H .

Algorithm 3 `bridgedTree` (s)

1. Call `basicTree&Path` (s) and let T be the tree returned. If T is a basic tree, return. Otherwise, T consists of two vertex-disjoint basic trees joined by a bridge-path, and proceed to the following steps.
 2. Initialize `midAs` to {all the mid-nodes in T }.
 3. While `midAs` $\neq \emptyset$ do
 - (a) Pick $a \in \text{midAs}$.
 - (b) If there exists an unvisited a -node a' in $\Gamma(a)$ then
 - (i) All the d -nodes in $\Gamma(a')$ must be unvisited, and there must exist at least two such nodes; Pick one, say d' .
 - (ii) Extend T from a by attaching a 2-edge (a, a', d') .
 - (iii) Mark both a' and d' as “visited”.
 - (iv) Call `basicTree` (d') and let T' be the basic tree returned.
 - (v) Set `midAs` $\leftarrow \text{midAs} \cup \{a'\} \cup \{\text{all the mid-nodes in } T'\}$.
 - (vi) Attach T' to T .
 - (c) Else remove a from `midAs`.
 4. Return T .
-

Lemma 1 *The tree T returned by `bridgedTree` (s) is a maximal 2-tree of type either basic or bridged.*

Proof When T is a bridged tree, it is explicitly checked if any unvisited a -node exists adjacent to each node of current T , and whenever found, it becomes visited and a part of T . Therefore, there cannot remain an unvisited a -node adjacent to T when it is returned.

When T is a basic tree, all the unvisited a -nodes adjacent to any d -node of T must have been explicitly checked, and hence, such an a -node outside of T cannot be

adjacent to any d -node of T . The output T being a basic tree implies that no bridge-path was found to emanate from every d -node of T . So, if an unvisited a -node exists outside of T , it cannot be adjacent to any a -node of T as the existence of such a node would imply that of a bridge-path emanating from some d -node in T . $\square$

4.2 Forest Construction

Assuming again that H is a connected component of $G'[A \cup D]$, we describe how to build a forest $\mathcal{F}_H$ in H so that all the nodes in $A(H)$ are spanned by the trees in $\mathcal{F}_H$. A call of `bridgedTree`($\cdot$) returns a maximal 2-tree T , but it may not span $A(H)$ entirely because of structural constraints imposed on T . We therefore construct a forest $\mathcal{F}_H$ of trees, basic and bridged, in H by repeatedly calling `bridgedTree`($\cdot$), so that every a -node in H becomes a part of some tree in it. We do these operations paying attention to an additional property to be imposed on $\mathcal{F}_H$. We say that a forest $\mathcal{F}$ is *compact* if the set of vertices of the trees in it induces a connected subgraph in G' .

If a forest $\mathcal{F}_H$ is compact, all the trees in it can be connected together using a minimum number of edges, and such $\mathcal{F}_H$ can be easily computed by restarting `bridgedTree`($\cdot$) every time from a node adjacent to an already computed tree.

Algorithm 4 `buildForest`(H)

1. Initialize the forest $\mathcal{F}_H$ to be empty, and set $i = 1$.
 2. Pick any d -node r_1 in H .
 3. While there exists a node in $A(H)$ not spanned by any tree in $\mathcal{F}_H$ do
 - (a) Call `bridgedTree`(r_i), let T_i be the tree returned, and add T_i to $\mathcal{F}_H$.
 - (b) Pick an unvisited r_{i+1} adjacent to some T_j for $1 \leq j \leq i$ if it exists, and increment i .
 4. Output $\mathcal{F}_H$.
-

Lemma 2 *Let $T_1, T_2, \dots, T_k$ be the trees of the forest $\mathcal{F}_H$ constructed by `buildForest`(H). Then,*

- $\mathcal{F}_H$ is compact.
- If an edge (u, v) exists between T_i and T_j (i.e., $u \in V(T_i)$ and $v \in V(T_j)$),
 - exactly one of u and v is an a -node and the other is a d -node.
 - if $u \in A$ then $i < j$, and otherwise, $i > j$.

Proof It is straightforward to observe the following:

- 1) r_i belongs to T_i , and 2) there exists j ($1 \leq j < i$) and an edge (a_{j_i}, r_i) in H s.t. the a -node a_{j_i} belongs to T_j , for all i ($1 \leq i \leq k$). It follows that $\mathcal{F}_H$ is compact as all of the T_i 's can be connected together by those $(k - 1)$ edges.

- The tree T_1 is maximal within H , and when H_i denotes the graph remaining after all the vertices of earlier constructed T_j 's ($1 \leq j \leq i - 1$) are removed from H , T_i is maximal within H_i , for all i ($1 \leq i \leq k$).

Thus, any node remaining unvisited after T_i is constructed must be a d -node if it is adjacent to T_j for some $1 \leq j \leq i$, and if no such nodes exist in the last step of this algorithm, no unvisited a -node can remain in H , and hence, the while-loop terminates immediately when this occurs. $\square$

4.3 Forest Modification

Let $\mathcal{F}_C$ denote a forest of trees each spanning a distinct component of $G[C]$, and $\mathcal{F}_H$ the forest computed by `buildForest`(H) for a connected component H of $G'[A \cup D]$. Then, the trees in $\mathcal{F}_H$'s for all the components H of $G'[A \cup D]$, together with those in $\mathcal{F}_C$ and those trees each of which spanning a nontrivial component of $G[D]$ can be seen to constitute a forest $\mathcal{F}$ satisfying Properties 1 through 3. Such $\mathcal{F}$ may not satisfy Property 4, however, and this subsection describes how to modify $\mathcal{F}_H$'s so that $\mathcal{F}$ satisfies all of them.

Clearly all the nodes in $A(H) \cup C$ are spanned by the trees in $\mathcal{F}_H$ and $\mathcal{F}_C$. Moreover, $\mathcal{F}_H$ is a compact forest as already observed. As will be treated in the next section, one can satisfy Property 4 by assigning dual values within $E[T]$ s.t.

$$y(E[T]) \geq \frac{|E(T)| + 1}{4} \quad \text{and} \quad y(\delta(u)) \leq \frac{1}{2}, \quad \forall u \in V(T)$$

when T is either a tree spanning a component in $G[C]$, a bridged tree in $G'[A \cup D]$, or a basic tree in $G'[A \cup D]$ containing a fat d -node. In a case where T is a basic tree with no fat d -nodes in it, on the other hand, it may no longer be possible to assign dual values in $E[T]$ satisfying all of these bounds². Yet it will be shown (in the next section) how to assign dual values in $E[T]$ so that y is feasible when restricted to within $E[T]$ and $y(E[T]) \geq (|E(T)| + 1)/4$ for a basic tree T whereas $y(\delta(u))$ is not necessarily bounded by $1/2$, $\forall u \in V(T)$. A problem here is that such y may not yield a dual feasible solution when extended to a whole $y \in \mathbb{R}^E$ as there could be a node $u \in V(T)$ with $y(\delta(u)) > 1/2$ and an edge (u, v) with $v \notin V(T)$. To avoid such a situation, therefore, we consider using basic trees of a more specific type.

Let T' be a subtree of a basic tree T in $\mathcal{F}_H$. We say that T' is *internally closed* if none of the a -nodes of T' is adjacent to another tree in $\mathcal{F}_H \cup \mathcal{F}_C$ or to a d -node in $D(T) - D(T')$. A basic tree $T \in \mathcal{F}_H$ containing an internally closed tree as its

²Think of the case of $H \simeq K_{s,s+1}$ for instance, which can appear as a subgraph in $G[A \cup D]$. Since any basic tree T spanning H consists of s many 2-edges, $|E(T)| = 2s$ and hence, $(|E(T)| + 1)/4 = (2s + 1)/4$. Suppose $y(\delta(u)) \leq 1/2$, $\forall u \in V(H)$. Then, $y(E[T]) \leq \sum_{u \in A(H)} y(\delta(u)) \leq s/2$. Thus, $y(E[T]) < (|E(T)| + 1)/4$ in this case.

sub 2-tree is called *internal* whereas it is *external* if every nonempty sub 2-tree is not internally closed. Observe that

- if no inter-tree edge is incident to $A(T)$ for a basic tree $T \in \mathcal{F}_H$, T itself is internally closed (and hence, internal), but T could be external if some inter-tree edge is incident to $A(T)$, and
- in the forest output by $\text{buildForest}(H)$, the last tree T_k constructed must be internally closed (and hence, internal as well), if it is basic and not adjacent to C , due to the property specified by Lemma 2.

Let $T_i \in \mathcal{F}_H$ be a basic tree rooted at r_i and introduce the natural ancestor-descendant relation among nodes in it for each T_i of $\mathcal{F}_H$. The following procedure tears apart T_i into parts, paste them to other trees in $\mathcal{F}_H \cup \mathcal{F}_C$ using external edges, and the only remnant of T_i is an internally closed 2-tree or otherwise, it is just a single node, which, being of no use for us, will be subsequently discarded.

Algorithm 5 $\text{resolveTree}(T_i)$

1. Given a basic tree T_i , if it is empty or internally closed, exit here.
 2. Otherwise, there must exist an inter-tree edge (a, u) s.t. $a \in A(T_i)$, and either $u \in D(T')$ for $T' \in \mathcal{F}_H$ or $u \in V(T')$ for $T' \in \mathcal{F}_C$.
 3. Let (d_1, a, d_2) be the 2-edge of T_i into which (a, u) is incident, and let d_1 be closer to the root of T_i than d_2 .
 4. Replace the 2-edge (d_1, a, d_2) of T_i by (d_1, a, u) .
 5. By the operation of the previous step, T_i is divided into two, and the one containing the root is immediately attached to T' via edge (a, u) . The other is a sub 2-tree of T_i , which is basic by itself, and this tree is named anew as T_i rooted at d_2 .
 6. Recursively call $\text{resolveTree}(T_i)$.
-

Observation 1 *In case that T' referred to in Step 2 of $\text{resolveTree}(T_i)$ is a tree in $\mathcal{F}_H$, it must have been constructed by $\text{buildForest}(H)$ later than T_i because of the existence of the inter-tree edge (a, u) (Lemma 2).*

Let $\mathcal{F}_H = \{T_1, T_2, \dots, T_k\}$ be the forest computed by $\text{buildForest}(H)$. We'd like to eliminate from $\mathcal{F}_H$ any external basic tree having no fat d -node, and an obvious approach is to apply $\text{resolveTree}(T_i)$ to such T_i 's. Recall that $\mathcal{F}_H$ computed by $\text{buildForest}(H)$ is a compact forest, and a call of $\text{resolveTree}(T_i)$ could result in removal of a d -node from T_i , and hence, the compactness of $\mathcal{F}_H$ could be lost if $\text{resolveTree}(\cdot)$ is applied to trees in it in an arbitrary order. To avoid this, $\mathcal{F}_H$ will be modified by the next procedure, using Observation 1, by applying $\text{resolveTree}(T_i)$ to each of external basic trees without fat d -nodes $T_i \in \mathcal{F}_H$ in the reversal of the order they were created by $\text{buildForest}(H)$.

Algorithm 6 $\text{modifyForest}(H)$

-
1. For $i = k - 1$ downto 1 do
 - (a) If T_i is a basic tree with no fat d -node in it and not internally closed, call $\text{resolveTree}(T_i)$.
 - (b) If T_i is a single node, remove it from $\mathcal{F}_H$.
 2. return $\mathcal{F}_H$.
-

Lemma 3 *The forest $\mathcal{F}_H$ computed by $\text{modifyForest}(H)$ has the following properties:*

1. Every tree in $\mathcal{F}_H$ is either a basic tree or a bridged tree, and every basic tree is internal if it does not contain a fat d -node.
2. Let $\mathcal{F}'$ denote the set of trees in $\mathcal{F}_C$ involved when $\text{resolveTree}(T_i)$ was applied during $\text{modifyForest}(H)$. Then, $\mathcal{F}_H \cup \mathcal{F}'$ is compact.

Proof 1. If T_i is a basic tree without a fat d -node and not yet internally closed, then $\text{resolveTree}(T_i)$ is called by which T_i becomes either empty or internally closed. Therefore, every basic tree remaining in $\mathcal{F}_H$ must be internal or otherwise, it must contain a fat d -node.

2. Recall r_1 denotes the root of T_1 in $\mathcal{F}_H$. To see that $\mathcal{F}_H \cup \mathcal{F}'$ is compact, it suffices to show that every node in it can reach r_1 by passing only the nodes of trees belonging to $\mathcal{F}_H \cup \mathcal{F}'$. Before running $\text{modifyForest}(H)$, $\mathcal{F}_H$ was compact. To be more precise, every node of T_i can reach r_1 using only the nodes of $T_1, \dots, T_i$. Consider now the run of $\text{modifyForest}(H)$, and suppose $\text{resolveTree}(T_i)$ is applied to T_i . At this point of time any of T_1 through T_{i-1} has been kept intact, and application of $\text{resolveTree}(T_i)$ can affect the reachability with r_1 only for the nodes in T_i through T_k . It cannot, however, if all the nodes of T_i remain in $\mathcal{F}_H \cup \mathcal{F}'$ after application of $\text{resolveTree}(T_i)$, and thus we consider now the case when a node u of T_i disappears from any tree of $\mathcal{F}_H \cup \mathcal{F}'$ as a result of $\text{resolveTree}(T_i)$. It must be the case then that u was a leaf d -node of T_i before $\text{resolveTree}(T_i)$ was applied to T_i . Then, any other node of T_i can be seen to remain reachable from r_1 even after removal of u , but so must be any node of trees T_{i+1} through T_k because no edge can exist in the first place between u and those trees (Lemma 2). $\square$

4.4 Overall Algorithm

Now that all of the component procedures have been presented, the entire algorithm, which we refer to as C2EDS, can be described as follows:

1. Compute the Gallai-Edmonds decomposition (A, C, D) of G .

2. Compute a forest $\mathcal{F}_C$ of trees each spanning every component of $G[C]$.
3. Shrink every nontrivial component of $G[D]$ into a fat d -node and denote the resulting graph by G' .
4. For each connected component H of $G'[A \cup D]$ do
 - (a) Construct a forest $\mathcal{F}_H$ by running `buildForest` (H).
 - (b) Restructure $\mathcal{F}_H \cup \mathcal{F}_C$ by running `modifyForest` (H).
5. For each fat d -node d in G' do
 - (a) Let H be the component of $G'[A \cup D]$ containing d , and T_d be a spanning tree of the original component of $G[D]$ shrunk into d .
 - (b) If d belongs to $T \in \mathcal{F}_H \cup \mathcal{F}_C$, modify T by inserting T_d into T in place of d ,
 - (c) Else (i.e., d does not belong to any tree in $\mathcal{F}_H \cup \mathcal{F}_C$), add T_d to $\mathcal{F}_H$.
6. Let $\tilde{\mathcal{F}}$ be the forest of all the trees in $\mathcal{F}_C$ and $\mathcal{F}_H$'s for all the components H of $G'[A \cup D]$. Compute and output a tree by minimally connecting all the trees in $\tilde{\mathcal{F}}$.

Lemma 4 *The forest $\tilde{\mathcal{F}}$ obtained in Step 6 of C2EDS is a compact forest consisting of the following types of trees, where, in cases 1, 2 and 5, any fat d -node d in G' is replaced by a spanning tree of the original component shrunk into d :*

1. a tree T spanning a component of $G[C]$, with possibly some number of basic trees T' attached to T , where exactly one leaf u of T' is superimposed with a node of T ,
2. a bridged tree in $G'[A \cup D]$,
3. an internal basic tree in $G'[A \cup D]$ with no fat d -node in it,
4. a tree spanning a nontrivial component of $G[D]$,
5. a basic tree in $G'[A \cup D]$ containing a fat d -node.

Proof It is clear from Lemma 3.1 and the way $\mathcal{F}_C$ and $\mathcal{F}_H$'s are constructed in G' that each tree of $\tilde{\mathcal{F}}$ is of one of the types listed above. It remains to show that $\tilde{\mathcal{F}}$ is compact in G . Shrink every spanning tree of a nontrivial component in $G[D]$ to a fat d -node in $\tilde{\mathcal{F}}$. It follows from Lemma 3.2 and the fact that $A \cup C$ is spanned by $\tilde{\mathcal{F}}$ that it is compact in G' . But then, when every fat d -node is replaced back to a spanning tree of the original component in $G[D]$, the resulting $\tilde{\mathcal{F}}$ must be compact in G . $\square$

Tight Instances Consider a graph G consisting of a cycle C_n on vertices $x_1, x_2, \dots, x_n$, and edges $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$. Clearly, G has a perfect matching and all the vertices belong to C in the Gallai-Edmonds decomposition of G . The algorithm C2EDS, when applied to G , computes a tree spanning all the vertices (i.e., a spanning tree of G) and outputs it, whereas the cycle on $x_1, x_2, \dots, x_n$ is an optimal C2-EDS solution and the path $\langle y_1, x_1, x_2, \dots, x_n, y_n \rangle$ is an optimal 2-TC solution.

A similar tight instance can be constructed by attaching two edges (x_i, y_i) and (x_i, y'_i) to each vertex x_i of the cycle C_n . The Gallai-Edmonds decomposition then divides the vertices into $D = \{y_1, y'_1, y_2, y'_2, \dots, y_n, y'_n\}$ and $A =$

$\{x_1, x_2 \dots, x_n\}$. The algorithm this times computes a maximal bridged tree spanning all of x_i 's and exactly one of y_i and y'_i for each i , whereas the optimal solutions used in the previous example remain optimal.

5 Analysis

To see that the tree T output by the algorithm C2EDS of Section 4.4 is indeed a connected 2-eds, it suffices to verify that each vertex of G not belonging to T is non-adjacent to a leaf of T . Observe that any vertex u of G not spanned by T must occur in D as a singleton component of $G[D]$. Thus, u is adjacent only to nodes in A , and none of them can be a leaf of T as every a -node is of degree at least 2 in T . Therefore,

Lemma 5 *The tree computed by the algorithm C2EDS is a 2-edge dominating set for G .*

The remainder of the paper is devoted to an analysis of the approximation performance of the algorithm. For a matching M in a graph H , $y \in \mathbb{R}^{E(H)}$ is called an M -dual if $y_e = 1/2$ for each $e \in M$ and $y_e = 0$ for each $e \notin M$. Furthermore, y is called an M -dual respecting u for $u \in V(H)$ if M leaves u unmatched. Easily, an M -dual y is dual feasible for any matching M and $y(\delta(u)) = 0$ if y is an M -dual respecting u . For any attachment of a basic tree T' to an existing tree T where a leaf u of T' is superimposed with a node of T , we use an M' -dual $y_{T'}$ respecting u for the dual assignment on T' , where M' is any maximum matching in T' respecting u . Then, because M' matches all the nodes of T' but u , $y_{T'}$ by itself has the objective value of $|E(T')|/2$, enough for accounting for the size of T' , and such an attachment can be dropped from further consideration as its dual would not interfere with the one on T when assigned as described in the proof of the next lemma. It thus suffices to show how to assign duals for those trees T listed in Lemma 4, ignoring the existence of any attachment of basic trees.

Lemma 6 *For each tree T listed below, there exists dual feasible $y \in \mathbb{R}^{E(T)}$ with $y(E[T]) \geq (|E(T)| + 1)/4$, where, in cases 1, 2 and 5, any fat d -node d in G' is replaced by a spanning tree of the original component shrunk into d :*

1. a tree spanning a component of $G[C]$,
2. a bridged tree in $G'[A \cup D]$,
3. an internally closed basic tree in $G'[A \cup D]$ with no fat d -node in it,
4. a tree spanning a nontrivial component of $G[D]$,
5. a basic tree in $G'[A \cup D]$ containing a fat d -node.

Moreover, $y(\delta(u)) \leq 3/4$ if u is an a -node in an internally closed basic tree in $G'[A \cup D]$ and $y(\delta(u)) \leq 1/2$ for all the other nodes u in G .

Proof For any fat d -node d in G' , the original component B of $G[D]$ shrunk into d is factor-critical (Theorem 2.1), and suppose d appears as a node of some tree T

computed by C2EDS. When duals are assigned by an M -dual for a matching M on T , only one edge e of M is incident to d . Let u denote the unique node of B into which e is incident when d is expanded back to original B . It is assumed in what follows that dual values are assigned within B by an M_B -dual for a perfect matching M_B in $B - \{u\}$, for any of such d -nodes contained in the following trees of cases 1, 2 and 5, excluding the fat d -node d' referred to in case 5.

1. A tree spanning a component B of $G[C]$.

Let M_B denote a perfect matching for B , and let y be an M_B -dual. Then, $y(E[T]) = |V(B)|/4 = (|E(T)| + 1)/4$ for any spanning tree T for B .

2. A bridged tree in $G'[A \cup D]$. Observe that there exists a matching in any 2-tree T of size $|E(T)|/2$, one edge per 2-edge, while any bridged tree T' has a perfect matching M in it; a matching consisting of two edges from the bridge-path and a half of the edges from each of the 2-trees attached to the bridge-path. It thus follows by considering an M -dual for T' that there exists dual feasible $y \in \mathbb{R}^{E(T')}$ such that $y(E(T')) \geq (|E(T')| + 1)/4$.
3. An internally closed basic tree in $G'[A \cup D]$ with no fat d -node in it.

For an internally closed basic tree T , one can assign more duals, within the component induced by $V(T)$, than an M -dual on T can assert.

Lemma 7 *Let T be a basic tree and B be the component induced by $V(T)$. There exists $y \in \mathbb{R}^{E(B)}$, feasible within B , such that $y(E(B)) \geq (|E(T)| + 1)/4$ and $y(\delta(u)) \leq 1/2$ for each $u \in D(B)$.*

Proof Pick a leaf u of T , which is necessarily a d -node, as a root of T . Let us assume for a moment that there exists an edge in B between u and every a -node of T whether it actually exists or not. Let a_k be the a -node adjacent to u within T , and let $a_1, a_2, \dots, a_{k-1}$ be a sequence of other a -nodes in T in such an order that a_i comes before a_j (i.e., $i < j$) whenever a_i is a descendant of a_j in T .

An edge may or may not exist between u and a_i for any $i \leq k-1$, but we assume they exist (temporally) for all $i \in \{1, \dots, k\}$ (So, some of them are virtual and only the rest are real ones). Initially, the duals are assigned as follows. Set

$$y(a_i, u) = \frac{1}{2k} \quad \text{for each } i \in \{1, \dots, k\}.$$

Let M be a maximum matching for T respecting u , and assign

$$y_e = \frac{1}{2} - \frac{1}{4k} \quad \text{for each } e \in M.$$

The total value assigned is thus

$$k \cdot \left(\frac{1}{2k} + \frac{1}{2} - \frac{1}{4k} \right) = \frac{k}{2} + \frac{1}{4}.$$

Observe now that $y(\delta(a_i)) = \frac{1}{2} + \frac{1}{4k}$ for each a -node a_i and $y(\delta(d)) = \frac{1}{2}$ if $d = u$ and for all the other d -nodes d , $y(\delta(d)) = \frac{1}{2} - \frac{1}{4k}$. So, for any existing edge e in B , $y(\delta(e)) \leq 1$, and hence, y is dual feasible (Note: It might appear

that $y(\delta((u, a_i))) = \frac{1}{2} + \frac{1}{4k} + \frac{1}{2} > 1$ but in this case, $y_{(u, a_i)}$ is doubly counted, and the actual value is $\frac{1}{2k}$ less, that is $y(\delta((u, a_i))) = 1 - \frac{1}{4k}$.

The assigned dual values are repeatedly relocated by processing the edges (a_i, u) in the order of $i = 1, \dots, k-1$ as follows: Let a'_i denote the grandparent of a_i and d' be the d -node at the middle of the a_i - a'_i path. If the edge (a_i, u) is a real one and does exist, do nothing and go to the next i ($= i+1$). Otherwise, it is virtual and (a_i, u) does not exist in B . Then, remove it together with $y_{(a_i, u)}$ after adding the current value of $y_{(a_i, u)}$ to each of $y_{(a_i, d')}$ and $y_{(a'_i, u)}$ and also subtracting the same amount from $y_{(d', a'_i)}$ (Notice that the edge (d', a'_i) belongs to M). The alteration of y_e -values here can be seen to occur along the edges of the cycle $C_i = (a_i, d', a'_i, u)$ alternatively getting increased and decreased by a uniform amount. Repeat this until a_{k-1} is processed.

Observe first that each of processing a_i 's does not alter the total amount of duals because each increases some twice while it also decreases the same amount twice. Hence, $y(E(B)) = \frac{k}{2} + \frac{1}{4}$ at the end. The same goes to the values of $y(\delta(v))$ for each node v lying on C_i (and of course no changes in $y(\delta(\cdot))$ for any node not involved in C_i). Therefore, $y(\delta(e))$ remains the same if e is not lying entirely within C_i . It is not the case for the edges on C_i , and the value of $y(\delta((d', a'_i)))$ goes up (the other increasing $y(\delta(e))$ is for $e = (u, a_i)$ but it disappears when (u, a_i) is disposed of). The value of $y(\delta((d', a'_i)))$, initially being equal to $\frac{1}{2} - \frac{1}{4k}$, is increased at most once during the entire process, and the largest amount by which it can be increased is $(k-1) \times \frac{1}{2k} = \frac{1}{2} - \frac{1}{2k}$, and hence, $y(\delta((d', a'_i)))$ never exceeds 1. Moreover, subtractions from y_e can occur at most once for each $e \in M$ and nowhere else, and the largest amount that can be subtracted from is $\frac{1}{2} - \frac{1}{2k}$. Therefore, y_e with $e \in M$ could go down to

$$\left(\frac{1}{2} - \frac{1}{4k}\right) - \left(\frac{1}{2} - \frac{1}{2k}\right) = \frac{1}{4k}$$

but never goes below zero. Therefore, y remains dual feasible throughout. $\square$

4. A tree spanning a nontrivial component B of $G[D]$.

A factor-critical graph is known to have the following *odd ear decomposition*, where a path or cycle is called *odd* when its length (i.e., the number of edges) is odd:

Theorem 3 (see [28]) *Every factor-critical graph can be represented as $P_0 + P_1 + \dots + P_k$ where P_0 is a single vertex and for each i , P_{i+1} is an odd path having only its two endvertices in common with $P_0 + \dots + P_i$ or P_{i+1} is an odd cycle with precisely one vertex in common with $P_0 + \dots + P_i$.*

So, $P_0 + P_1$ is an odd cycle, and set $y_e = 1/4$ for each edge of this cycle. Let M_i denote a maximal matching within P_i for $i \geq 2$ such that the end-vertices of P_i are left unmatched by M_i . Then, $|M_i| = (|P_i| - 1)/2$, and set $y_e = 1/2$ for each edge of M_i . The sum of y_e 's for all the edges in B can be seen to be $|V|/4$,

and hence, $y(E(B)) = (|E(T)| + 1)/4$ for any tree T spanning B . It should be also noted that $y(\delta(u)) \leq 1/2$ for any $u \in V(B)$.

Every fat d -node is unshrunk into a nontrivial component B of $G[D]$ in Step 5 of the algorithm C2EDS. Such a d -node is either a part of some tree in $\mathcal{F}_H$'s or not, and in the latter case we use the dual assignment specified above. There is another case where duals are assigned as above, and it will be described next.

5. A basic tree containing a fat d -node.

Suppose T is a basic tree in $G'[A \cup D]$ containing a fat d -node d' . Let y_T denote the M -dual where M is the maximum matching for T respecting d' . For the nontrivial component B' of $G[D]$ into which d' is unshrunk, let T' be any spanning tree for B' and let $y_{B'}$ denote the dual assignment on $E(B')$ specified in the previous item (of “A tree spanning a nontrivial component B of $G[D]$ ”). For any fat d -node d'' contained in T other than d' , consider the corresponding component B'' of $G[D]$ and a spanning tree T'' for B'' . As exactly one node u in B'' is matched by M , there exists a matching M'' in $E(B'')$ perfectly matching all the nodes in B'' but u , and it can be seen that $y_T + y_{B'}$ added with the M'' -dual $y_{B''}$ can remain dual feasible. It then follows that the total duals thus assigned within $B = G[V(T) \cup V(B') \cup V(B'')]$ is $|E(T)|/4 + |V(B')|/4$ added with $(|V(B'')| - 1)/4$, that is

$$y(E(B)) = \frac{|E(T)| + (|E(T')| + 1) + |E(T'')|}{4} = \frac{|E(T) \cup E(T') \cup E(T'')| + 1}{4}.$$

This way, even if T contains more fat d -nodes in $G'[A \cup D]$, letting $\tilde{T}$ denote T with all of them replaced by the corresponding spanning trees for the components into which they become unshrunk, it can be seen there exists dual feasible y defined on $G[V(\tilde{T})]$ such that $y(E(B)) = (|E(\tilde{T})| + 1)/4$.

□

By Lemma 6 there is a way $y \in \mathbb{R}^{E[T]}$ to assign dual values within $E[T]$ for each tree $T \in \tilde{\mathcal{F}}$ s.t. $y(E[T]) \geq (|E(T)| + 1)/4$. Let y denote the combination of all the dual assignments involved. It can be verified that $y(\delta(e)) \leq 1$ for each $e \in E$ as follows. Certainly, it is the case when $e \in E[T]$ for some $T \in \tilde{\mathcal{F}}$ as y restricted to $E[T]$ is dual feasible. For $e \notin E[T]$ for any $T \in \tilde{\mathcal{F}}$, recall that $y(\delta(u)) \leq 1/2$ for all $u \in V$ unless u is an a -node in an internally closed basic tree. Since e is not incident to an a -node of any internally closed basic tree, $y(\delta(e)) \leq 1$ holds. Therefore, y thus determined is dual feasible as a whole, and since the final tree output by the algorithm is constructed by minimally connecting all the trees in the compact forest $\tilde{\mathcal{F}}$, the number of additional edges used is one less than the number of the trees in it. The size of the tree output is thus bounded by $4 \sum_{e \in E} y_e - 1 = 2 \cdot z_D(y) - 1 \leq 2 \cdot \text{dual}_2(G) - 1$. This completes the proof of Theorem 1.

Acknowledgements The authors are grateful to the anonymous referees for a number of valuable comments and suggestions.

References

1. Alber, J., Betzler, N., Niedermeier, R.: Experiments on data reduction for optimal domination in networks. *Ann. Oper. Res.* **146**, 105–117 (2006)
2. Arkin, E.M., Halldórsson, M.M., Hassin, R.: Approximating the tree and tour covers of a graph. *Inform. Process. Lett.* **47**, 275–282 (1993)
3. Armon, A.: On Min-max r -gatherings. *Theor. Comput. Sci.* **412**(7), 573–582 (2011)
4. Baker, B.S.: Approximation algorithms for NP-complete problems on planar graphs. *J. ACM* **41**, 153–180 (1994)
5. Berger, A., Fukunaga, T., Nagamochi, H., Parekh, O.: Approximability of the capacitated b -edge dominating set problem. *Theory Comput Syst.* **385**(1–3), 202–213 (2007)
6. Binkele-Raible, D., Fernau, H.: Enumerate and Measure: Improving Parameter Budget Management. In: *Proceedings International Conference on Parameterized and Exact Computation*, pp. 38–49. Springer, Heidelberg (2010)
7. Blum, J., Ding, M., Thaeler, A., Cheng, X.: *Connected Dominating Set in Sensor Networks and MANETs*. Springer, New York (2005)
8. Chellali, M., Favaron, O., Hansberg, A., Volkmann, L.: k -domination and k -independence in graphs: a survey. *Graphs Combin.* **28**, 1–55 (2012)
9. Chlebík, M., Chlebíková, J.: Approximation hardness of edge dominating set problems, vol. 11 (2006)
10. Cooper, C., Klasing, R., Zito, M.: Dominating Sets in Web Graphs. In: *Algorithms and Models for the Web-Graph*, LNCS 3243, pp. 31–43. Springer, Heidelberg (2004)
11. Dai, F., Wu, J.: On Constructing k -connected k -dominating Set in Wireless Ad Hoc and Sensor Networks. *J. Parallel Distrib. Comput.* **66**(7), 947–958 (2006)
12. Du, D.-Z., Wan, P.-J.: *Connected Dominating Set: Theory and Applications*. Springer, New York (2013)
13. Du, H., Ding, L., Wu, W., Kim, D., Pardalos, P.M., Willson, J.: Connected Dominating Set in Wireless Networks. In: Pardalos, P.M., Du, D.-Z., Graham, R. (eds.) *Handbook of Combinatorial Optimization*, pp. 783–833. Springer, New York (2013)
14. Escoffier, B., Monnot, J., Paschos, V.T.h., Xiao, M.: New results on polynomial inapproximability and fixed parameter approximability of edge dominating set. *Theory comput Syst.* **56**(2), 330–346 (2015)
15. Fernau, H.: EDGE DOMINATING SET: Efficient Enumeration-based Exact Algorithms. In: *Proceedings 2Nd International Conference on Parameterized and Exact Computation*, pp. 142–153. Springer, Heidelberg (2006)
16. Fernau, H., Fomin, F.V., Philip, G., Saurabh, S.: The Curse of Connectivity: t -Total Vertex (Edge) Cover. In: *Proceedings 16Th Annual International Conference on Computing and Combinatorics*, pp. 34–43. Springer, Heidelberg (2010)
17. Fernau, H., Manlove, D.F.: Vertex and edge covers with clustering properties: Complexity and algorithms. *J. Discrete Algorithms* **7**(2), 149–167 (2009)
18. Fink, J.F., Jacobson, M.S.: n -Domination in Graphs. In: Alavi, Y., Chartrand, G., Lick, D.R., Wall, C.E., Lesniak, L. (eds.) *Graph Theory with Applications to Algorithms and Computer Science*, pp. 283–300. John Wiley & Sons, Inc., New York (1985)
19. Fink, J.F., Jacobson, M.S.: On N -Domination, N -Dependence and Forbidden Subgraphs. In: Alavi, Y., Chartrand, G., Lick, D.R., Wall, C.E., author Lesniak, L. (eds.) *Graph Theory with Applications to Algorithms and Computer Science*, pp. 301–311. John Wiley & Sons, Inc., New York (1985)
20. Fomin, F.V., Gaspers, S., Saurabh, S., Stepanov, A.A.: On two techniques of combining branching and treewidth. *Algorithmica* **54**(2), 181–207 (2009)
21. Fujito, T.: On Matchings and b -Edge Dominating Sets: a 2-Approximation Algorithm for the 3-Edge Dominating Set Problem. In: *Algorithm Theory–SWAT 2014*, LNCS 8503, pp. 206–216. Springer, Cham (2014)
22. Gao, X., Zou, F., Kim, D., Du, D.-Z.: The Latest Researches on Dominating Problems in Wireless Sensor Network. In: *Handbook on Sensor Networks*, Pp. 197–226. World Scientific (2010)
23. Harary, F.: *Graph theory*. Addison-wesley, reading MA (1969)
24. Haynes, T.W., Hedetniemi, S.T., Slater, P.J.: *Domination in Graphs, Advanced Topics*. Marcel Dekker, New York (1998)
25. Haynes, T.W., Hedetniemi, S.T., Slater, P.J.: *Fundamentals of Domination in Graphs*. Marcel Dekker, New York (1998)

26. Hunt III, H.B., Marathe, M.V., Radhakrishnan, V., Ravi, S.S., Rosenkrantz, D.J., Stearns, R.E.: A Unified Approach to Approximation Schemes for NP- and PSPACE-Hard Problems for Geometric Graphs. In: Proceedings 2Nd Ann. European Symp. on Algorithms, LNCS 855, pp. 424–435. Springer (1994)
27. Kim, D., Gao, X., Zou, F., Du, D.-Z.: Construction of Fault-Tolerant Virtual Backbones in Wireless Networks. In: Handbook on Security and Networks, pp. 488–509. World Scientific (2011)
28. Lovász, L., Plummer, M.D.: Matching Theory. North-Holland, Amsterdam (1986)
29. Małafiejski, M., Żyliński, P.: Weakly Cooperative Guards in Grids. In: Proceedings 2005 International Conference on Computational Science and Its Applications - Volume Part I, pp. 647–656. Springer, Heidelberg (2005)
30. Savage, C.: Depth-first search and the vertex cover problem. Inf. Process. Lett. **14**(5), 233–235 (1982)
31. Schmied, R., Viehmann, C.: Approximating edge dominating set in dense graphs. Theor. Comput. Sci. **414**(1), 92–99 (2012)
32. Shang, W., Wan, P., Yao, F., Hu, X.: Algorithms for minimum m -connected k -tuple dominating set problem. Theor. Comput. Sci. **381**(1–3), 241–247 (2007)
33. Shi, Y., Zhang, Y., Zhang, Z., Wu, W.: A greedy algorithm for the minimum 2-connected m -fold dominating set problem. J. Comb. Optim. **31**(1), 136–151 (2016)
34. Thai, M.T., Zhang, N., Tiwari, R., Xu, X.: On approximation algorithms of k -connected m -dominating sets in disk graphs. Theor. Comput. Sci. **385**(1–3), 49–59 (2007)
35. Wu, Y., Li, Y.: Construction Algorithms for K -Connected M -Dominating Sets in Wireless Sensor Networks. In: Proceedings 9Th ACM International Symposium on Mobile Ad Hoc Networking and Computing, pp. 83–90. ACM, New York (2008)
36. Xiao, M., Kloks, T., Poon, S.-H.: New parameterized algorithms for the edge dominating set problem. Theor. Comput. Sci. **511**, 147–158 (2013)
37. Yannakakis, M., Gavril, F.: Edge dominating sets in graphs. SIAM J. Appl. Math. **38**(3), 364–372 (1980)
38. Zhou, J., Zhang, Z., Wu, W., Xing, K.: A greedy algorithm for the fault-tolerant connected dominating set in a general graph. J. Comb. Optim. **28**(1), 310–319 (2014)